

Chapter 6 Sample Problems

1. Effective Access Time example: A computer has a single cache (off-chip) with a 2 ns hit time and a 98% hit rate. Main memory has a 40 ns access time. What is the computer's effective access time? If we add an on-chip cache with a .5 ns hit time and a 94% hit rate, what is the computer's effective access time? How much of a speedup does the on-chip cache give the computer?

Answers:

$$2 \text{ ns} + .02 * 40 \text{ ns} = 2.8 \text{ ns.}$$

With the on-chip cache, we have $.5 \text{ ns} + .06 * (2 \text{ ns} + .02 * 40 \text{ ns}) = .668 \text{ ns}$. The speedup is $2.8 / .668 = 4.2$.

2. Virtual memory problem: Assume a computer has on-chip and off-chip caches, main memory and virtual memory. Assume the following hit rates and access times: on-chip cache 95%, 1 ns, off-chip cache 99%, 10 ns, main memory: X%, 50 ns, virtual memory: 100%, 2,500,000 ns. Notice that the on-chip access time is 1 ns. We do now want our effective access time to increase much beyond 1 ns. Assume that an acceptance effective access time is 1.6 ns. What should X be (the percentage of page faults) to ensure that EAT is no worse than 1.6 ns?

Answer: $EAT = 1\text{ns} + .05 * (10 \text{ ns} + .01 * (50 \text{ ns} + (1 - X) * 2,500,000 \text{ ns}))$. Since we want EAT to be no more than 1.6 ns, we solve for X with $1.6 \text{ ns} = 1\text{ns} + .05 * (10 \text{ ns} + .01 * (50 \text{ ns} + (1 - X) * 2,500,000 \text{ ns}))$. $X = 1 - (((((1.25 \text{ ns} - 1 \text{ ns}) / .05) - 10 \text{ ns}) / .01) - 50 \text{ ns}) / 2,500,000$. $X = 0.99994 = 99.994\%$. Our miss rate for virtual memory must be no worse than .006%!

3. Cache/Memory Layout: A computer has an 8 GByte memory with 64 bit word sizes. Each block of memory stores 16 words. The computer has a direct-mapped cache of 128 blocks. The computer uses word level addressing. What is the address format? If we change the cache to a 4-way set associative cache, what is the new address format?

Answers:

With 8 GB and a 64 bit word size, there are $8 \text{ GB} / (8 \text{ bytes} / \text{word}) = 1 \text{ GW}$ of memory. This requires 30 bits for an address. Of the 30 bits, we need 4 bits for the word on the line and 7 bits for the block number, leaving $30 - (7 + 4) = 19$ bits for the tag. So the address format is 19 – 7 – 4.

If we have a 4-way set associative cache instead, then there will be 4 sets with $128 / 4 = 32$ blocks per set. So we would only need 5 bits for the block number, leaving $30 - (5 + 4) = 21$ bits for the tag. So the address format is 21 – 5 – 4.

4. Direct Mapping Question: Assume a computer has 32 bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache would we look for each of the following addresses? Addresses are given in hexadecimal for convenience.
 - a. 1A2BC012
 - b. FFFF00FF
 - c. 12345678
 - d. C109D532

Answers: Of the 32 bit address, the last four bits denote the word on the line. Since four bits is used for one hex digit, the last digit of the address is the word on the line. With 256 blocks in the cache, we need 8 bits to denote the block number. This would be the third to last and second to last hex digit.

- a. this would be block 01, which is block 1
- b. this would be 0F which is block 15
- c. this would be 67 which is block 103 (remember, 67 is a hex value)
- d. this would be 53 which is block 83.

5. Assume a program consists of 8 pages and a computer has 16 frames of memory. A page consists of 4096 words and memory is word addressable. Currently, page 0 is in frame 2, page 4 is in frame 15, page 6 is in frame 5 and page 7 is in frame 9. No other pages are in memory. Translate the memory addresses below.
- a. 111000011110000
 - b. 000000000000000

Answers: The first three bits denote the page number (8 pages) and we swap it out for the four bit frame number (16 frames).

- a. 111 (page 7) becomes 1001 (frame 9) so 111000011110000 becomes 1001000011110000
- b. 000 (page 0) becomes 0010 (frame 2) so 000000000000000 becomes 0010000000000000